

Relatório Comparativo

Kelvin de Lima Rodrigues, Raquel Maciel Coelho de Sousa, Ulisses Bonfim, Vinícius Menezes Monte

Abstract—Este relatório apresenta uma análise comparativa entre três padrões de arquitetura frontend — MVC, MVP e MVVM — aplicados em aplicações React. O estudo avalia o comportamento do frontend quando integrado a dois modelos de backend: REST (pull) e reativo (push/event-driven). A análise prioriza a separação de responsabilidades, a clareza do fluxo de dados entre camadas e o impacto prático da reatividade na sincronização entre clientes.

Index Terms—Arquitetura de Software, React, MVC, MVP, MVVM, REST, Sistemas Reativos

I. INTRODUÇÃO

ESTE trabalho compara o impacto do modelo de backend — REST (pull) versus reativo (push/event-driven) — sobre um frontend React implementado em três padrões: MVC, MVP e MVVM. A análise foca na separação de responsabilidades, na clareza do fluxo de dados entre camadas e no efeito da reatividade na consistência observada entre clientes.

II. ANÁLISE DAS ARQUITETURAS

A. Onde ficou a maior parte da lógica em cada arquitetura

A maior parte da lógica do fluxo (orquestração de casos de uso, integração com gateway e decisão de quando mutar o estado do *store*) permaneceu fora dos componentes React. A View ficou predominantemente responsável por renderização e captura de eventos, com variações discretas no MVVM devido ao consumo explícito de resultados de sucesso/falha.

1) **MVC (Model-View-Controller):** **Maior parte da lógica:** no *TaskController*. O Controller centraliza a orquestração: chama o *TaskGateway* (REST ou reativo) e aplica mutações no *TaskStore*. A View (React) permanece essencialmente declarativa, disparando ações do Controller e renderizando o estado exposto pelo *store*.

2) **MVP (Model-View-Presenter):** **Maior parte da lógica:** no *TaskPresenter*. O Presenter atua como coordenador do fluxo, mantendo a View em postura passiva: a interface delega ações ao Presenter e reflete o estado observado no *store*. Na implementação, MVC e MVP ficaram muito próximos na distribuição de responsabilidades: muda o nome/contrato do coordenador, mas a centralização do fluxo permanece.

3) **MVVM (Model-View-ViewModel):** **Maior parte da lógica:** no *TaskViewModel*. O ViewModel concentra a coordenação das operações (load/create/update/delete), incorpora validações e retorna resultados tipados (ex.: *VmResult/VmError*), tornando sucesso e falha explícitos. A View participa um pouco mais ao consumir esses resultados para decisões de UI (ex.: fechar modal apenas em *ok*, exibir mensagem em erro), mas a coordenação do fluxo continua centralizada fora do React.

B. Observação comum às três abordagens

Independentemente do padrão, uma parcela relevante da complexidade ficou na infraestrutura compartilhada: um *TaskStore* observável e a ponte entre eventos reativos e mutações de estado. Ao encapsular o comportamento *push* nessa camada, as diferenças práticas entre MVC, MVP e MVVM passam a depender menos de *como* o backend entrega mudanças e mais de *como* cada padrão organiza contratos, validações e tratamento de erros.

III. INTEGRAÇÃO COM BACKEND

A. Qual arquitetura foi mais simples de integrar com o backend reativo

Na implementação, **não houve diferença material** de facilidade entre as arquiteturas, pois a reatividade foi **encapsulada na infraestrutura**.

O contrato *TaskGateway* expôs a assinatura de eventos (via *subscribe*), e a seleção entre *RestTaskGateway* e *Real-timeTaskGateway* ficou restrita ao *wiring* (ex.: *createGateway*). Além disso, o *TaskStore* traduziu eventos recebidos do gateway em mutações determinísticas (*upsert*, *removeById*), mantendo a interface sincronizada automaticamente. Como resultado, *TaskController*, *TaskPresenter* e *TaskViewModel* permaneceram focados em orquestração de casos de uso, sem acoplamento a streams/eventos.

B. O que mudou no frontend ao trocar REST por reativo

A mudança principal ocorreu em **configuração e montagem** (*wiring*), não na estrutura arquitetural.

O que mudou:

- O gateway migrou de *pull* (HTTP/REST) para *push* (eventos em tempo real), ativando *subscribe*.
- O *TaskStore* passou a ser alimentado continuamente por eventos (ex.: *upsert/removeById*), reduzindo a necessidade de “recarregar” para refletir mudanças.
- A View continuou reagindo ao *store* via observabilidade (ex.: *useSyncExternalStore*), recebendo atualizações automaticamente.

O que não mudou:

- As responsabilidades centrais de *TaskController/TaskPresenter/TaskViewModel* permaneceram: orquestrar ações e lidar com resultados.
- Os componentes React não precisaram incorporar lógica de stream/eventos; a UI permaneceu declarativa e orientada ao estado.

Em resumo, a troca REST → reativo ficou concentrada no *gateway* e no *wiring* do *store*, preservando o desenho das camadas e mantendo semelhante o custo de adaptação entre MVC, MVP e MVVM.

TABLE I
RESUMO COMPARATIVO (OBSERVADO NA IMPLEMENTAÇÃO)

Padrão	Ponto central	Erros/contratos	Troca p/ reativo
MVC	Controller	Implícito/leve	Isolada em gateway/store/wiring
MVP	Presenter	Implícito/leve	Isolada em gateway/store/wiring
MVVM	ViewModel	Explícito (resultados)	Isolada em gateway/store/wiring

IV. METODOLOGIA DE COMPARAÇÃO

A análise foi conduzida como uma comparação controlada no nível de arquitetura. Foi utilizado o *mesmo* Todo App (mesmo domínio, componentes de UI e mecanismo de estado local via *TaskStore*), variando apenas: (i) o padrão arquitetural do frontend (MVC, MVP, MVVM) e (ii) o modo de integração com o backend (REST e reativo). Em cada combinação, duas instâncias do frontend foram executadas simultaneamente para observar consistência e propagação de mudanças entre clientes.

No modo REST (*pull*), a instância secundária só reflete alterações após uma ação explícita de atualização (nova requisição/*refresh*). No modo reativo (*push*), as mudanças são entregues como eventos via *subscribe* e convertidas em mutações determinísticas no *TaskStore*, mantendo a interface sincronizada.

A. Critérios adotados

A avaliação priorizou aspectos verificáveis no código e no comportamento observado durante a execução:

- **Distribuição de responsabilidades:** localização da orquestração (casos de uso) e do controle de mutações de estado.
- **Acoplamento com React:** dependência do fluxo crítico em hooks/componentes versus isolamento em classes/adapters.
- **Tratamento de erros:** explicitude do contrato de sucesso/falha e padronização de mensagens/estados.
- **Custo de troca REST → reativo:** abrangência das alterações (gateway/store/wiring versus camada de apresentação).
- **Manutenibilidade:** legibilidade do fluxo e risco de dispersão de regras de fluxo na View em cenários de escala.

V. LIMITAÇÕES E AMEAÇAS À VALIDADE

- **Escopo reduzido:** o Todo App é deliberadamente simples e não cobre cenários comuns de escala (ex.: múltiplos módulos, permissões, cache).
- **Isolamento de variabilidade (efeito de uniformização):** ao encapsular REST versus reativo no *TaskGateway* e no *TaskStore*, a integração reativa deixou de diferenciar fortemente MVC/MVP/MVVM. Esse é um resultado arquitetural positivo, mas reduz o contraste nesse eixo.
- **Viés de implementação:** decisões locais (ex.: formato do *store*, assinatura de *subscribe*, retorno tipado de

operações) podem favorecer determinados estilos (especialmente MVVM) por construção.

- **Métrica de latência informal:** a noção de latência “imperceptível” foi avaliada por observação durante a demonstração, sem instrumentação de métricas (logs/telemetria) para quantificação.

VI. CONCLUSÃO CRÍTICA

A. Qual arquitetura escolher para um sistema maior? Por quê?

Para um sistema de maior porte, o **MVVM** foi a alternativa mais adequada.

Motivo central: o MVVM favorece contratos explícitos de operação (sucesso/falha) e centraliza o fluxo em um ponto evolutivo (*TaskViewModel*), reduzindo a probabilidade de lógica de fluxo e tratamento de erro migrarem para componentes React conforme o sistema cresce.

Por que não MVC/MVP como primeira opção: na implementação, MVC e MVP permaneceram muito semelhantes: um coordenador central (*TaskController* ou *TaskPresenter*) com tratamento de erro mais implícito e pouca propagação formal de falhas para a View. Em sistemas maiores, isso tende a empurrar decisões de UI (feedbacks, estados intermediários, tentativas, mensagens) para dentro da camada React, aumentando complexidade incidental.

Por que MVVM escala melhor:

- Contratos tipados de resultado (ex.: *VmResult/VmError*) tornam o fluxo determinístico na UI (ex.: fechar modal apenas em *ok*).
- Tratamento sistemático de erros e pontos de observabilidade (logs/métricas) ficam concentrados no *ViewModel*, padronizando comportamento.
- A View se mantém predominantemente declarativa, consumindo estado e resultados sem conhecer detalhes do *gateway/store*.

Por fim, a integração com backend reativo não foi o fator decisivo, pois foi uniformizada pela infraestrutura. A decisão pelo MVVM se apoia principalmente em manutenibilidade e evolução do frontend.

AGRADECIMENTOS

Os autores agradecem ao IFCE e ao professor César Olavo pelo suporte acadêmico e orientação durante o desenvolvimento deste trabalho.